Johan Camilo Murcia – Esteban Alejandro Hernandez

Robótica Móvil y Sistemas Autónomos.

Informe Proyecto Robótica 2.

Introducción:

El proyecto 2 de Robótica móvil y sistemas autónomos tiene como objetivo resolver problemas de planificación de caminos geométricos y relocalización en una configuración inicial q0 y una configuración final qf. Generando una solución mediante el uso de herramientas como ROS2 y Gazebo.

Planificación:

2.1 Descripción del método

El método implementado es **A* sobre un C-space discretizado en cuadrícula**, complementado con suavizado por visibilidad directa. Se compone de tres etapas:

Etapa 1 — Construcción del C-space

El espacio de trabajo se discretiza en celdas cuadradas de 0.25 m × 0.25 m. El robot de dimensiones 18 × 21 cm se aproxima como un cuadrado de 30 cm de lado, lo que permite tratarlo como un punto al inflar los obstáculos por ROBOT_RADIO = 0.15 m. Cada celda se clasifica según su distancia mínima a cualquier obstáculo o pared:

Clasificación	Condición	Costo
Libre	d_min ≥ 2 × 0.15 m	normal
Semi-libre	0.15 m ≤ d_min < 0.30 m	× 2
Ocupada	d_min < 0.15 m	no transitable

Etapa 2 — A* 8-conectado

Sobre la cuadrícula se ejecuta A* con 8 direcciones de movimiento (4 cardinales + 4 diagonales). La función de costo es:

- Movimiento cardinal: 1.0
- Movimiento diagonal: √2
- Celda semi-libre: costo × 2 para preferir rutas alejadas de obstáculos
- Heurística: distancia euclidiana a la celda meta

Si el inicio o la meta caen en celda ocupada, se usa BFS para encontrar la celda libre más cercana.

Etapa 3 — Suavizado por visibilidad directa

Se aplica un algoritmo greedy usando trazado de línea de Bresenham: desde el waypoint actual se busca el waypoint más lejano al que se puede llegar en línea recta sin cruzar celdas ocupadas. Esto reduce el número de segmentos innecesarios.

2.2 Resultado del método

El resultado es una secuencia de waypoints (x, y) en coordenadas mundo, que se convierte al formato:

<q0, qrot0-1, q1, qrot1-2, q2, ..., qn, qrotn>

Donde cada qroti-i+1 tiene orientación θ = atan2(Δy, Δx) del segmento siguiente, y qrotn = qf con la orientación final requerida. Esto se guarda línea a línea en Camino-EscenaX.txt.

2.3 Traducción a comandos del robot

El nodo ejecuta una **máquina de estados** que recorre los segmentos:

ROTANDO → MOVIENDO → ROTANDO → MOVIENDO → ... → ROTACION_FINAL → RELOCALIZAR → DONE

- **ROTANDO:** control proporcional de velocidad angular. La velocidad se reduce proporcionalmente cuando el error angular es menor a 0.5 rad para evitar sobrepasar el objetivo. Se considera completo cuando |error| < TOL_ANGULAR.
- **MOVIENDO:** publica VEL_LINEAL = 0.3 m/s y mide el avance real con odometría. Para cuando distancia_recorrida ≥ distancia_segmento o si el LiDAR detecta obstáculo a menos de DIST_SEGURA = 0.25 m en cono frontal ±25°. Si se bloquea, replanifica desde la posición actual.
- **ROTACION_FINAL:** igual que ROTANDO pero hacia qf.θ.
- **RELOCALIZAR:** usa LiDAR para calcular q_act.

2.4 Tabla de resultados

Escena	qf = (x, y, θ)	qf-est = (x, y, θ)	Δ pos (m)	Δ ang (°)
1	(3.25, 4.25, 90.0°)	(2.8977, 3.2909, 80.94°)	1.0218	9.06
2	(3.25, 4.25, 90.0°)	(3.0609, 4.4785, 89.98°)	0.2966	0.02
3	(3.25, 4.25, 90.0°)	(3.2438, 4.2634, 88.62°)	0.0148	1.38
4	(3.25, 4.25, 90.0°)	(2.8413, 3.5673, 80.33°)	0.7957	9.67
5	(3.25, 4.25, 90.0°)	(2.7158, 4.2620, 81.48°)	0.5344	8.52
6	(3.25, 4.25, 90.0°)	(2.7498, 4.2744, 81.65°)	0.5008	8.35

2.5 Posibles mejoras

- **Mayor resolución:** reducir CELL_SIZE a 0.10 m para detectar pasillos estrechos y trazar trayectorias más precisas cerca de esquinas.
- **Replanificación dinámica:** ya implementada, pero podría mejorarse con detección anticipada de obstáculos usando el LiDAR antes de llegar al punto de bloqueo.
- **Control de posición en lazo cerrado:** reemplazar el control bang-bang actual por un PID de posición que use la odometría continuamente para corregir desviaciones durante el movimiento.
- **Suavizado con curvas:** usar splines o Bezier entre waypoints para evitar detenciones completas en cada punto intermedio.

Relocalización:

3.1 Descripción del método

Al arribar a la pose teórica q_f, el robot inicia una fase de **corrección de deriva posicional**. Para ello, captura dos lecturas de rango mediante el sensor LiDAR: la distancia frontal (d_f a 0°) y la lateral derecha (d_r a -90°), ambas referenciadas estrictamente al **marco de coordenadas del cuerpo (body frame)**.

El algoritmo proyecta estas lecturas al **marco de coordenadas global (world frame)** utilizando la orientación teórica θ _f. Se generan vectores unitarios ortogonales (f', r') para definir las direcciones de proyección y se ejecuta un proceso de **ray-casting** sobre el mapa conocido para obtener los puntos de intersección esperados (`front_pt`, `right_pt`).

Posteriormente, se establece un **sistema de ecuaciones lineales 2×2**. Este sistema vincula la posición real p_act con las restricciones geométricas impuestas por las distancias medidas y los productos punto de las proyecciones. La resolución se realiza mediante la **Regla de Cramer**, permitiendo despejar las coordenadas (x, y) con precisión milimétrica.

Finalmente, se realiza una actualización del estado del robot. Dado que este método se especializa en la **corrección de traslación**, la orientación θ _f se mantiene constante, asumiendo que la odometría o la IMU han mantenido la referencia angular estable, delegando al LiDAR la resolución exclusiva de la posición absoluta en el plano.

3.2 Causas de error de localización

- **Deslizamiento de ruedas:** durante giros y aceleraciones la odometría pierde precisión acumulando error en qf-est.
- **Resolución de cuadrícula:** los waypoints quedan en centros de celdas de 25 cm, no en la trayectoria óptima continua, desviando al robot desde el inicio.

- **Error angular acumulado:** un pequeño error en la orientación al inicio de cada segmento se traduce en error de posición proporcional a la distancia del segmento.
- **Ruido del LiDAR:** las lecturas d_f y d_r tienen ruido gaussiano que introduce error directamente en q_act.
- **Saltos de odometría:** el bridge de Gazebo puede publicar lecturas espurias en (0,0) que corrompen la estimación de posición interna.
- **Singularidad del sistema:** si det ≈ 0 (frente y derecha casi paralelos al mismo eje mundial), la solución es numéricamente inestable.

3.4 Tabla de resultados

Escena	qf-est = (x, y, θ)	q_act = (x, y, θ)	Δ pos (m)	Δ ang (°)
1	(2.8977, 3.2909, 80.94°)	(3.2200, 4.2452, 90.00°)	1.0073	9.06
2	(3.0609, 4.4785, 89.98°)	(3.2200, 4.4665, 90.00°)	0.1596	0.02
3	(3.2438, 4.2634, 88.62°)	(3.2200, 4.2509, 90.00°)	0.0269	1.38
4	(2.8413, 3.5673, 80.33°)	(3.2200, 4.3364, 90.00°)	0.8573	9.67
5	(2.7158, 4.2620, 81.48°)	(3.2200, 4.2405, 90.00°)	0.5047	8.52
6	(2.7498, 4.2744, 81.65°)	(3.2200, 4.2535, 90.00°)	0.4706	8.35

3.5 Análisis de resultados

- **Escena 3** tiene el menor error de posición (1.5 cm) — el camino con más waypoints intermedios paradójicamente fue el más preciso, ya que la replanificación corrigió la trayectoria.
- **Escena 1** tiene el mayor error (1.02 m) — camino directo de un solo segmento largo donde cualquier error angular inicial se amplifica.
- La **relocalización con LiDAR** corrige consistentemente la posición X (siempre devuelve x=3.22) pero no puede corregir Y porque el error angular acumulado en qf-est no se compensa.
- El **error angular** es idéntico entre las dos tablas en todas las escenas, confirmando que el LiDAR solo corrige posición, no orientación.

Anotaciones finales:

- Durante la realización de las pruebas se hicieron correciones constantes del codigo, ya que, a medida que aumentaba la complejidad de las pruebas mayor cantidad de errores aparecian.
- Se trato de mantener los mismos para metros de inicio durante todas las pruebas, tamaño de robot, de celda, etc., sin embargo, durante la quinta y sexta prueba se bajo

el tamaño del robot y se aumentó la tolerancia, ya que el algoritmo de replanificación encontraba bucles infinitos o puntos muertos donde no avanzaba la simulación.

- La exactitud de las pruebas varía de acuerdo con la cantidad de veces que se tiene que recalcular la planificación, ya que va a cumulando errores de posición con constancia, permitiendo que, indique que culmino con l atarea cuando se queda en el borde de esta.

Videos:

Todos los videos se encuentran en el siguiente link:

https://drive.google.com/drive/folders/1o5dQoUgB8jlU7eZTVtlZv4E53G5n3nbr?usp=drive_link